

Err and err and err again, but less and less and less.

Aula 20

Fecho das CFL para substituições e morfismos

Definição

Uma função $h : \Sigma \rightarrow \Gamma^*$ tal que $(\forall \sigma \in \Sigma)(h(\sigma) \text{ é uma CFL})$ diz-se uma **substituição livre de contexto**.

Teorema

Se L é uma CFL e h é uma substituição livre de contexto, então $h(L)$ é também uma CFL.

Demonstração: Seja $G = \langle V, \sigma, P, z_0 \rangle$ a gramática tal que $\mathcal{L}(G) = L$ e para cada $\sigma \in \Sigma$, seja $G_\sigma = \langle V_\sigma, \Gamma, P_\Sigma, z_\sigma \rangle$ tal que $\mathcal{L}(G_\sigma) = h(\sigma)$. Podemos supor, sem perda de generalidade que todas os conjuntos V e V_σ são disjuntos dois a dois. Então $h(L)$ é gerada pela CFG

$$G' = \left\langle V \cup \bigcup_{\sigma \in \Sigma} V_\sigma, \Gamma, P', z_0 \right\rangle,$$

em que P' contém todas as produções de todos P_σ mais todas as produções de P em que cada ocorrência de σ foi substituída por z_σ . É trivial que esta gramática G' é tal que $\mathcal{L}(G') = h(L)$. $\square$

Recorda que um morfismo é uma aplicação definida como a extensão natural de $h : \Sigma \rightarrow \Gamma^*$ a Σ^* , se $|h(\sigma)| = 1$, para todo o $\sigma \in \Sigma$.

Um exemplo: Seja L a linguagem gerada pela seguinte gramática:

$$S \rightarrow 0S0 \mid 1S1 \mid \varepsilon,$$

e seja $t : \{0, 1\} \rightarrow \{a, b\}^*$ em que $t(0)$ é a linguagem gerada por

$$S_0 \rightarrow aS_0b \mid b,$$

e $t(1)$ a linguagem gerada por

$$\begin{aligned} S_1 &\rightarrow AB, \\ A &\rightarrow aaA \mid b, \\ B &\rightarrow bBb \mid a. \end{aligned}$$

A linguagem $t(L)$ é, então, gerada pela gramática

$$\begin{aligned} S &\rightarrow S_0SS_0 \mid S_1SS_1 \mid \varepsilon, \\ S_1 &\rightarrow AB, \\ A &\rightarrow aaA \mid b, \\ B &\rightarrow bBb \mid a. \end{aligned}$$

Teorema

As CFL são fechadas para a imagem directa e recíproca de morfismos.

Demonstração: Que as CFL são fechadas para a imagem de morfismos é consequência directa do teorema anterior, pois os morfismos são, trivialmente, substituições independentes de contexto.

Quanto à imagem reversa de morfismos o problema é mais delicado. Seja $L = \mathcal{L}(A)$ uma CFL com alfabeto Γ , com A um PDA, e $h : \Sigma \rightarrow \Gamma^*$ um morfismo. Poderíamos ser tentados a definir um autómato $\mathcal{L}(A') = \mathcal{L}(h^{-1}(L))$ em Σ^* , como fizemos para o teorema equivalente para as linguagens regulares, e de cada vez que lemos um σ , calcular $h(\sigma)$ e obter o resultado em A para o transpor para A' . Mas como se tratam de PDAs, a evolução da pilha de A quando este lê $h(\sigma)$ (que tem potencialmente comprimento maior do que 1) pode não ser exprimível por uma só transição de A' . Assim temos que guardar num “buffer” os caracteres de $h(\sigma)$ para os podermos processar um de cada vez. Os PDAs não possuem memória para além da que podem simular no “controlo”, ou seja no conjunto de estados. Esta simulação de “memória” tem a limitação que tem que ser de uma dimensão constante, ou seja não pode depender do tamanho do input. Felizmente como Σ é finito, e cada imagem $h(\sigma)$ é constituída por uma só palavra, o conjunto

$$U = \bigcup_{\sigma \in \Sigma} \text{suf}(h(\sigma))$$

também é finito e portanto podemos simular o comportamento de um *buffer* no conjunto finito dos estados.

Suponhamos, então, que

$$A = \langle Q, \Gamma, \Lambda, \delta, q_0, z_0, F \rangle.$$

O novo PDA A' que vai gerar $h^{-1}(L)$ será

$$A' = \langle Q \times U, \Sigma, \Lambda, \delta', (q_0, \varepsilon), z_0, F' \rangle,$$

com, para todos os estados q ,

- i) $\delta'((q, \varepsilon), \sigma, y) = \{((q, h(\sigma)), y)\}$
- ii) $\delta'((q, x), \varepsilon, y) = \{((p, x), \gamma) \mid (p, \gamma) \in \delta(q, \varepsilon, y)\},$
- iii) $\delta'((q, \sigma x), \varepsilon, y) = \{((p, x), \gamma) \mid (p, \gamma) \in \delta(q, \sigma, y)\},$

e $F' = F \times \{\varepsilon\}$.

Temos agora que mostrar que $h^{-1}(\mathcal{L}(A)) = \mathcal{L}(A')$. Primeiro observemos que se $\alpha q h(\sigma) \vdash_A^* \beta p$ então, pela aplicação das regras i), ii) e iii)

$$\alpha(q, \varepsilon)\sigma \vdash_{A'} \alpha(q, h(\sigma)) \vdash_{A'}^* \beta(p, \varepsilon).$$

Portanto, se A aceita $h(w)$, i.e.,

$$z_0 q_0 h(w) \vdash_A^* \beta p$$

para algum $p \in F$ e $\beta \in \Lambda^*$, então

$$z_0(q_0, \varepsilon)w \vdash_{A'}^* \beta(p, \varepsilon)$$

e, portanto A' aceita w . Portanto $\mathcal{L}(A') \supseteq h^{-1}(\mathcal{L}(A))$.

Para a inclusão no sentido contrário, suponhamos que A' aceita $w = \sigma_1 \sigma_2 \cdots \sigma_n$. Como a regra i) só pode ser aplicada quando o *buffer* está vazio, a sequência de movimentos que leva à aceitação de w tem que ser

$$\begin{aligned} z_0(q_0, \varepsilon)\sigma_1 \sigma_2 \cdots \sigma_n & \vdash_{A'}^* \alpha_1(p_1, \varepsilon)\sigma_1 \sigma_2 \cdots \sigma_n, \\ & \vdash_{A'} \alpha_1(p_1, h(\sigma_1))\sigma_2 \sigma_3 \cdots \sigma_n, \\ & \vdash_{A'}^* \alpha_2(p_2, \varepsilon)\sigma_2 \sigma_3 \cdots \sigma_n, \\ & \vdots \\ & \vdash_{A'}^* \alpha_n(p_{n-1}, \varepsilon)\sigma_n, \\ & \vdash_{A'} \alpha_n(p_{n-1}, h(\sigma_n)), \\ & \vdash_{A'}^* \alpha_{n+1}(p_n, \varepsilon), \end{aligned}$$

com $p_n \in F$. As transições de (p_i, ε) para $(p_i, h(\sigma_i))$ faz-se pela regra i), as outras transições pelas regras ii) e iii). Portanto $z_0 q_0 \vdash_A^* \alpha_1 p_1$, e, para todo o i ,

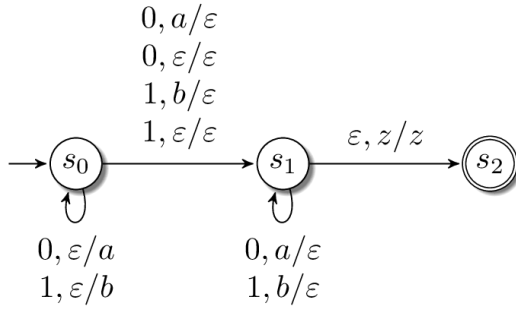
$$\alpha_i p_i h(\sigma_i) \vdash_A^* \alpha_{i+1} p_{i+1}.$$

Tomando estes movimentos em conjunto, e porque sendo h um morfismo $h(\sigma_1 \sigma_2 \cdots \sigma_n) = h(\sigma_1)h(\sigma_2) \cdots h(\sigma_n)$, temos

$$z_0 q_0 h(\sigma_1 \sigma_2 \cdots \sigma_n) \vdash_A^* \alpha_{n+1} p_n,$$

donde $h(\sigma_1 \sigma_2 \cdots \sigma_n) \in \mathcal{L}(A)$, e $\mathcal{L}(A') \subseteq h^{-1}(\mathcal{L}(A))$. Portanto concluímos que $\mathcal{L}(A') = h^{-1}(\mathcal{L}(A))$. $\square$

Um exemplo: Seja $\Sigma = \{0, 1\}$ e $h : \Sigma^* \rightarrow \Sigma^*$ o morfismo definido por $h(0) = 10$ e $h(1) = 1$. Consideremos a CFL L com alfabeto Σ definida pelo seguinte PDA:



Construamos, então, o PDA que decide $h^{-1}(L)$. Construíamos primeiro

$$U = \text{suf}\{h(\sigma) \mid \sigma \in \Sigma\} = \{10, 0, 1, \varepsilon\}.$$

O PDA terá, então, como conjunto de estados $\{s_0, s_1, s_2\} \times \{10, 0, 1, \varepsilon\}$, $\{(s_2, \varepsilon)\}$ como conjunto de estados finais e (q_0, ε) como estado inicial.

A função δ' é definida como

$$\begin{aligned}
 \delta'((s_0, \varepsilon), 0, \varepsilon) &= \{((s_0, 10), \varepsilon)\}, \\
 \delta'((s_0, \varepsilon), 1, \varepsilon) &= \{((s_0, 1), \varepsilon)\}, \\
 \delta'((s_0, 10), \varepsilon, \varepsilon) &= \{((s_0, 0), b), \varepsilon, \varepsilon\}, \\
 \delta'((s_0, 10), \varepsilon, b) &= \{((s_1, 0), \varepsilon)\}, \\
 \delta'((s_0, 0), \varepsilon, \varepsilon) &= \{((s_0, \varepsilon), a), ((s_1, \varepsilon), \varepsilon)\}, \\
 \delta'((s_0, 0, \varepsilon), a) &= \{((s_1, \varepsilon), \varepsilon)\}, \\
 \delta'((s_0, 1), \varepsilon, \varepsilon) &= \{((s_0, \varepsilon), b), ((s_1, \varepsilon), \varepsilon)\}, \\
 \delta'((s_0, 1), \varepsilon, b) &= \{((s_1, \varepsilon), \varepsilon)\}, \\
 \delta'((s_1, \varepsilon), 0, \varepsilon) &= \{((s_1, 10), \varepsilon)\}, \\
 \delta'((s_1, \varepsilon), 1, \varepsilon) &= \{(s_1, 1), \varepsilon\}, \\
 \delta'((s_1, \varepsilon), \varepsilon, z) &= \{(s_2, \varepsilon), z\}, \\
 \delta'((s_1, 10), \varepsilon, b) &= \{((s_1, 0), \varepsilon)\}, \\
 \delta'((s_1, 0), \varepsilon, a) &= \{(s_1, \varepsilon), \varepsilon\}, \\
 \delta'((s_1, 1), \varepsilon, b) &= \{(s_1, \varepsilon), \varepsilon\}.
 \end{aligned}$$

Fecho das CFL para com a intersecção com linguagens regulares

Apesar das CFL não serem fechadas para a intersecção, são fechadas para a intersecção com linguagens regulares.

Teorema

Seja L_1 uma CFL e L_2 uma linguagem regular, então $L_1 \cap L_2$ é uma CFL.

Demonstração: Seja $A = \langle Q_1, \Sigma, \Gamma, \delta_1, q'_0, Z_0, F_1 \rangle$ um PDA que represente a linguagem L_1 e $B = \langle Q_2, \Sigma, \delta_2, q''_0, F \rangle$ um DFA que represente a linguagem L_2 . Então $L_1 \cap L_2$ é reconhecida pelo seguinte PDA:

$$\langle Q_1 \times Q_2, \Sigma, \Gamma, \delta, (q'_0, q''_0), Z_0, F_1 \times F_2 \rangle,$$

em que

$$\delta((q_1, q_2), a, X) = \{((q'_1, q'_2), \alpha) \mid ((q'_1, \alpha) \in \delta_1(q_1, a, X)) \wedge (\delta_2(q_2, a) = q'_2)\}.$$

A construção é a construção do autómato produto, já feita para demonstração que as linguagens regulares são fechadas para a intersecção, mas com o pormenor de se ter que preservar o funcionamento da pilha. O autómato resultante reconhece, trivialmente, a linguagem pretendida. $\square$

Quociente de uma CFL por uma linguagem regular

Recordemos que, dadas duas linguagens L_1 e L_2 define-se

$$L_1 / L_2 = \{x \mid \exists y \in L_2, xy \in L_1\}.$$

Teorema

Seja L_1 uma CFL e L_2 uma linguagem regular, então L_1 / L_2 é uma CFL.

Demonstração: Seja $A = \langle Q_1, \Sigma, \Gamma, \delta_1, q'_0, Z_0, F_1 \rangle$ um PDA que represente a linguagem L_1 e $B = \langle Q_2, \Sigma, \delta_2, q''_0, F \rangle$ um DFA que represente a linguagem L_2 . O PDA que reconhece este quociente tem que proceder em duas fases: numa primeira lê o input x e processa-o no PDA A , e numa segunda tenta, especulativamente, encontrar palavras que possam estar em L_2 e que concatenadas a x formem uma palavra de L_1 .

Assim um PDA que reconheça L_1 / L_2 é

$$\langle Q_1 \times (Q_2 \cup \{\perp\}), \Sigma, \Gamma, \delta', (q'_0, \perp), Z_0, F_1 \times F_2 \rangle.$$

A definição de δ é a seguinte:

- As transições correspondentes à primeira fase são aquelas a que correspondem os estados da forma $(q_1, \perp)$. Assim para quaisquer $q_1 \in Q_1, a \in \Sigma \cup \{\varepsilon\}$ e $X \in \Gamma$

$$\delta((q_1, \perp), a, X) = \{((q_2, \perp), \alpha) \mid (q_2, \alpha) \in \delta_1(q_1, a, X)\}.$$

- Em qualquer estado $(q_1, \perp)$ o input pode terminar e portanto têm que existir transições que passem para o funcionamento correspondente à segunda fase. Assim para todo o $q \in Q_1$ e $X \in \Gamma$

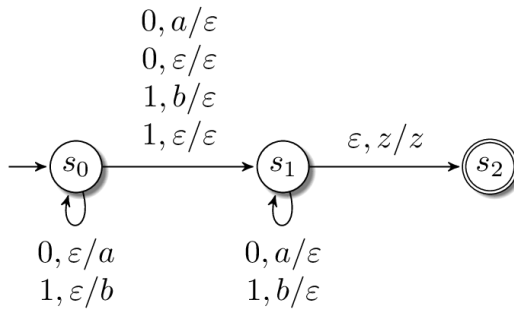
$$\delta((q, \perp), \varepsilon, X) = \{((q, q_0''), X)\}.$$

- Finalmente, na segunda fase, procuramos, não deterministicamente, palavras de $y \in L_2$ que possam levar que $xy \in L_1$.

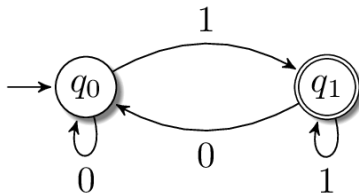
$$\delta((q_1, q_2), \varepsilon, X) = \{((q_1', q_2'), \alpha) \mid \delta_2(q_2, a) = q_2' \wedge \delta_1(q_1, a, X) \ni (q_1', \alpha)\}.$$

Dada a a forma como foi construído o autómato, é trivial que este reconhece L_1 / L_2 . $\square$

Um exemplo: Seja L_1 a linguagem definida pelo PDA seguinte:



e L_2 definido pelo seguinte DFA



O PDA que define o quociente L_1 / L_2 é então

